Untitled

Preprint, compiled May 11, 2026

1 Paper v2: outline (draft 2)

Working title: **Production Transformers as Universal Computers at Scale: 128-Digit Decimal Multiplication via In-Context Substrate Design**

Successor to `paper-paperese.md` (May 2024, Claude 3 Opus, 256 ECA rules including Rule 110, and 27-bit binary multiplication). The May 2026 result demonstrates the same core thesis — that a production transformer can be elicited to function as a universal computer — at a scale where "universal computation" is no longer a theoretical badge granted by simulating Rule 110 on a small tape, but is an empirical operating regime: thousands of compositional steps, products of $\sim 10^{255}$ magnitude demonstrated end-to-end, with $\sim$ 100% measured accuracy at the planned 64-digit benchmark, on a production model, no fine-tuning, no calculator.

The contribution is not a shift of focus away from universal computation. It is the opposite: a sharper realization of it, with an account of which structural properties of the tape design make the substrate behave as the unbounded-tape primitive a Turing machine requires.

A second framing carries through the entire paper: **what is conventionally called "reasoning" in current LLM discourse — chain-of-thought, scratchpads, intermediate steps — is at root indistinguishable from running a computation step by step.** The output stream is the computation. There is no "behind the scenes" symbolic reasoning happening separately from the emitted tokens; the tokens are the computation. For algorithms that are **computationally irreducible** (Wolfram 2002) — multi-digit multiplication is the canonical case — no shortcut exists. The atomic steps must be performed in sequence. The question is not whether the model can "reason about" the answer but whether it can faithfully **execute** the irreducible algorithm step by step. The substrate properties we identify are not mechanisms for better reasoning; they are the conditions under which a transformer's forward pass can serve as the transition function of a computationally irreducible process.

1.1 Headline result (vs. v1)

The earlier paper's framing — production transformer as empirical universal computer via ICL — extends naturally. What is new in v2:

1. The scale at which the demonstration holds (~4000 compositional steps; 25× linear past published state of the art on the same task).
2. A substrate primitive (trim continuation with landmark slicing) that decouples problem size from context window — the trace itself is the unbounded tape, not the model's context.
3. A set of six identifiable substrate properties that together realize the Turing-machine-implementation requirements in an LLM substrate.
4. A failure-mode cascade — 12 distinct empirical observations about what the model can and cannot reliably compute per token — that constitutes the constructive evidence for those substrate properties.

Dimension	v1 (2024)	v2 (2026)	Δ
Operand size demonstrated	27-bit binary ($\sim$ 9 decimal digits)	**128-digit decimal** (existence proof)	$\sim$ 14× linear
Operand size measured ($n \geq 12$)	$\sim$ 27-bit	**64-digit decimal** ($n = 25$ planned)	$\sim$ 7× linear
Product size at largest demonstration	54-bit ($\sim$ 16 decimal digits)	**256-digit decimal**	$\sim$ 16× linear
Product magnitude at largest	$\sim 10^{16}$	$\sim$ **10^{255}**	$\sim 10^{239}$
Model	Claude 3 Opus	Claude Opus 4.6	newer family, two major revs
Continuation	Chain by appending output	Trim with landmark-anchored slicer	new substrate primitive
Substrate properties identified	1 (dense positional indexing)	**6** (formalized below)	new

---

1.2 Sections

1.2.1 Abstract / Introduction

The thesis is the same as v1: a production transformer trained by gradient descent on natural language, with no architectural modifications and no fine-tuning, can be elicited to function as a universal computer via in-context learning over patterned examples. v1 established this at small scale; v2 establishes it at a scale where the demonstration is no longer trivial.

The published landscape on decimal multiplication has moved substantially between v1 and v2 — see Related Work — but the substrate-design approach reported here exceeds all of it by a wide margin without specialized training. Specifically:

- General-purpose production models with conventional prompting: effective ceiling around 4–5 digit decimal multiplication.
- Fine-tuned or specialized-architecture approaches with custom embeddings, reversed digit order, or implicit-CoT training: 12–15 digit decimal multiplication at ~ 100% accuracy.
- This work: **128-digit decimal multiplication demonstrated end-to-end on Claude Opus 4.6**, no fine-tuning, no architectural modifications. At 128 digits the run completes successfully but not reliably — the trace is long enough that per-token failure probabilities compound, and only a fraction of attempts pass byte-for-byte. The headline accuracy benchmark is at **64 digits** ($n = 25$ trials planned, on a compressed-tape variant), where we expect ~ 100%.

That is roughly 4× linear past the best published reliable multiplication result for the 64-digit benchmark, plus a further 2× at 128 demonstrated as an existence proof. Both on a generalist production model rather than a specialized one. The mechanism is in-context substrate design, not training.

1.2.2 Method

General approach (continued from v1) Same methodology as v1: model receives a structured training tape of worked examples, is asked to execute the algorithm on a fresh input, and the output is matched byte-for-byte against a deterministic reference evaluator. No English explanation, no chain-of-thought scaffolding, no external tools. The model induces the algorithm from the patterned examples and executes it via autoregressive emission.

The algorithm — Tanton sliding-strip cross--multiplication Cross-multiplication of n-cell × n-cell operands (cells base-100, two decimal digits each). The Tanton sliding-strip reformulation reverses B into a tape R such that for each diagonal index k, the pair indices (i, t) both increase monotonically by 1 across the row. No per-pair $j = k - i$ subtraction. The R tape is pre-reversed in the user input — the model transcribes it during each REFRESH but never derives it.

The substrate as Turing-machine implementation A Turing machine requires: (a) an unbounded tape, (b) a finite state / transition function, (c) deterministic step semantics, (d) a halt condition. The six substrate properties identified here map directly to these requirements as they manifest in an LLM substrate:

1. **Deterministic single-direction trace.** Every emitted token is a function of tokens to its immediate left. The model's forward-pass forms the transition function over the local window; the trace forms the tape contents.
2. **Externalized counters.** Anything the model would need finite internal state to track across many forward passes — cycle counters, row indices, carry chains — is written explicitly into the trace as bounded counters (e.g. tick = $N/12$, $[i/i_{\text{last}}]$, carry = c). The tape carries state the model cannot.
3. **Memoization on bounded sub-operations.** For decimal at chunk-size 2, the model writes its own $A_i \times d$ lookup table once at trace start (T block of 64 rows × 10 entries). Per-leaf sub-products in the body of the trace are then lookups against what the model wrote. The table is regenerated, not pre-supplied; the model still performs every required 1×2 digit product — once, in a clean context — rather than thousands of times scattered across the trace.
4. **Cross-check equations on each computation.** Bare numeric emissions can slip silently; equations break visibly. Each computed value appears as part of an equation (`P1*10+P2=prod`, `total=carry*100+cell`) whose terms are independently visible on the line. Errors self-anchor.
5. **Trim continuation with landmark-anchored slicing.** On API response overflow, the assistant prefill is sliced to the most recent FIRE block whose REFRESH has completed, with the trace prelude (`CHUNK=2` line and the T table) preserved at the head under a `<HISTORY_TRUNCATED>` marker. Each continuation sends exactly one FIRE-window of context. The model's effective tape is unbounded by its context window. This is the substrate primitive that closes the v1 paper's open question: the model's 200k-token context window is not the binding constraint.
6. **Explicit halt condition.** A `DONE` token paired with an API- level stop sequence. Without it the model drifts into prose at end of trace.

Trim continuation, formally Each step of the algorithm (one k-row) writes its own situational summary at its start: `RESUME k=N tick=T/M FIRE|SKIP carry=C prev=O… iLast=L t0=X`. The slicer keeps everything from the most recent FIRE row whose REFRESH block has finished emitting; the prelude is preserved. On continuation the model receives:

```
CHUNK=2
T:
A0_..: 0|0 1|.. 2|.. … 9|..
```

```
…
A63_..: 0|0 1|.. … 9|..
<HISTORY_TRUNCATED>
RESUME k=<latest FIRE row> tick=0/12 FIRE …
[FIRE block]
[partial pair lines through cut point]
```

Two structural consequences:

- **Unbounded tape.** Total trace length is not bounded by the model's input/output context window; only one FIRE window plus the prelude must fit. For 128-digit operands at REFRESH interval 12, a FIRE window is on the order of 5K–10K tokens; well inside any modern context.
- **Bounded local context per step.** The model's per-call work is always a finite local window. This is the Turing-machine analogue precisely: a finite transition function operating over a fixed local read.

1.2.3 Implementation

Single open-source repository (this one); the harness is reusable across programs. Each program declares an `index.ts` (config + training inputs) and an `eval.ts` (reference evaluator). The runner streams the model's response and compares to the reference character-by-character via `checkRollingSolution`; on first divergence the run fails. Continuation is handled by the unified `testWithModel` function which routes any gateway slug (`anthropic/...`, `openai/...`) and applies provider-specific options inline. Trim/stack continuation modes, warm-start (`--from=K`), and stop sequences all work uniformly across providers.

1.2.4 Results

Demonstration at 128 digits 128-digit operands A and B, executed on Claude Opus 4.6 at temperature 0. A successful run produced a 256-digit product with byte-for-byte match against the reference trace; product magnitude $\sim 2.2\times10^{255}$. At 128 digits the run is **achievable but not yet reliable** — the trace is ~50–60K output tokens across ~10–15 chained API calls, and per-token failure probabilities compound over thousands of atomic emissions. The 128×128 result is reported as an **existence proof** for the substrate at this scale, not as a measured accuracy.

Accuracy benchmark at 64 digits The headline accuracy measurement is planned at **64 digits**: $n = 25$ trials on random 64-digit operand pairs, on a compressed- tape variant of the substrate, against the reference evaluator byte-for-byte. We expect $\sim$ 100% on this benchmark — the trace is ~10–15K output tokens, well inside the regime where the substrate properties keep every per-token decision in the model's reliable range. This is the comparison point against the published 12–15-digit fine-tuned frontier.

For reference:

The run cost on the order of $1–$3, output ~50–60k tokens spanning roughly 10–15 chained API calls. Cache reads dominate input after the first call; the system prompt and the trace prelude (with T) remain cache-resident throughout.

Quantity	Magnitude
Atoms in the observable universe	10^{80}
Legal chess positions (Shannon number)	$\sim 10^{40}$
Estimated distinct chess games	$\sim 10^{120}$
Planck volumes in the observable universe	$\sim 10^{183}$
$A \times B$ **in this run**	$\sim$ **10^{255}**

Failure-mode cascade The substrate properties were not designed top-down. They were extracted from a sequence of empirically observed failure modes, each revealing a load-bearing assumption about the model's per- token reliability. The cascade is itself a contribution: it constitutes evidence about what the model's effective transition function can and cannot compute, in a form that can be replicated and stressed at other scales.

Documented in order of discovery:

1. `pairs +1` increment slip at FIRE boundary → replaced with copy-friendly `iLast`.
2. `b.i` vs `a.i` parity ambiguity at row-end → one pair per line, uniform label.
3. k mod 16 FIRE/SKIP slip at deep k → externalized `tick=N/16` (later 12).
4. `START` / `CHUNK=` re-emission on resume → `<HISTORY_TRUNCATED>` marker preserves trace prelude in every continuation.
5. OUT delta vs cumulative semantics at deeper FIREs → range header `OUT 00..0<n-1>` + larger training example.
6. $2d\times2d$ leaf product slip ($87\times83 = 6921$) → model-written memoization table T at trace start; leaves become `<digit>|<product>` lookups.
7. Bare-combine slip ($1278 \rightarrow 781$) → explicit `P1*10+P2=prod` equation rather than bare numeric.
8. Carry-split slip ($8818 \rightarrow$ `c87` instead of `c88`) → chained `total=carry*100+cell` equation.
9. Bare-`prod` recap line drift between pair lines → uniform per-line shape with explicit `0+prod=prod` on first line.
10. End-of-trace prose drift → `DONE` token plus stop sequence.
11. $j = k - i$ per-pair subtraction error at large i → cross-slide variant with reversed-B (R) tape, both indices monotonic.
12. 64-cell tape transcription slip during REFRESH → hand R pre-reversed in user input; model only transcribes.

Each entry frames the failure mode at the token level: what specific decision the model slipped on, and what structural change in the tape made that slip impossible.

Cost, scaling, robustness The trace at 128×128 spans roughly 10–15 chunks; cache reads dominate input after the first call. Total cost is sub-$5 per run. The cost-per-output-digit is approximately constant across scales from 8×8 through 128×128, consistent with the substrate behaving as a fixed-overhead-per-step computation rather than one whose per-token cost grows with problem size.

1.2.5 Discussion

What v2 adds to the universal-computation thesis v1 demonstrated that the computational circuits necessary for universal computation are present in a production model — Rule 110 can be simulated, multiplication can be performed at small scale. The natural objection to v1 was that the demonstration relied on small tapes and short traces; the empirical case for universal computation rested on a Turing-completeness simulation rather than on long, compositional computation.

v2 closes that gap in two layers. At the planned 64-digit benchmark ($n = 25$), the substrate produces a multi-thousand-step compositional execution with ~ 100% measured accuracy across many chained API calls. At 128 digits the substrate produces a successful end-to-end run as an existence proof at the larger scale, though reliability is not yet measured at that depth. In both cases the substrate primitive (trim continuation) removes the context window as a binding constraint. The "universal computer" interpretation is no longer dependent on a Rule-110-style existence proof on a small tape; the production transformer is constructively executing long compositional algorithms whose state grows arbitrarily, on arbitrary inputs.

Failure modes as evidence about the model's transition function The 12 failure modes are empirical evidence about what the model's forward-pass can and cannot reliably compute as a single emitted token. Three classes emerge:

- **Reliable atomic operations.** Single-digit multiplication (1×2 digit results ≤ 891); 4-digit addition; literal transcription from in-context tapes; positional lookup against short structured labels. The model executes these at very high reliability — failures we observed in these classes occurred occasionally at deep k, not systematically.
- **Reliable bounded counters.** When externalized to the trace as small integers with explicit format, the model reliably increments them by 1 and applies bounded modular conditions (`tick=N/12 → FIRE` when `tick=0`). The same conditions evaluated implicitly against k alone fail at large k.
- **Unreliable open-ended operations.** Implicit running-sum recall across many lines; computing $j = k - i$ at each pair line for i in a large range; recomputing the row's pair count from k, N, M at every line. These are the failure surface that the substrate properties patch over.

The pattern: the model's per-token "circuit" can implement the atomic operations a Turing machine transition function requires (positional lookup, conditional, copy, single-digit arithmetic), but cannot reliably hold the unbounded state a Turing machine permits on its tape. The substrate's job is to keep state on the tape and let the model do per-token work that fits inside its reliable range.

The interface vs. the architecture (continued from v1) v1 argued that the compositional limit on multiplication reported by Dziri et al. (2024) characterized the interface between task representation and attention, not the architecture itself. v2 strengthens this claim by extending the operand size by another order of magnitude on the same architecture family, using the same in-context-learning mechanism, with no fine-tuning. The published multiplication frontier has advanced substantially in the intervening period (Lee et al. 2024 reaches 12–15 digit multiplication via reversed-digit + custom embeddings; Deng et al. 2024 reaches 9×9 with implicit-CoT training on GPT-2 Small; Yu et al. 2025 reverse-engineers the failure modes), but none of these approaches operate on production generalist models without specialization. The substrate-design approach reaches scales out of reach for those approaches, with the trade-off that it requires careful per-task tape design and runs at production-model output cost.

Reasoning as intermediate computation; irreducibility The mainstream framing of LLM capability divides into "reasoning" (a privileged cognitive operation the model performs to arrive at an answer) and "execution" (mechanical token emission). v2 makes the case that this distinction is empty operationally. The output stream **is** the reasoning. The intermediate tokens **are** the computation. There is nothing happening "behind" them that the emitted trace fails to capture.

For computationally irreducible algorithms — those whose answer cannot be obtained without performing the atomic steps in sequence — the substrate-design approach is not an optimization over some other route to the answer. It is the only route. The model's job is not to find a clever shortcut to $A \times B$ for 128-digit A, B; no such shortcut exists. Its job is to execute the ~ 4000 atomic steps faithfully. That is what reasoning is for this class of problem, and that is what the substrate makes reliable at scale.

This reframing has practical implications for the field. The gap between "the model can multiply 3-digit numbers in one shot" and "the model can multiply 128-digit numbers via in-context substrate" is not a gap in the model's intelligence. It is a gap in the format of the request. Asking the model to emit the answer to a 128-digit multiplication directly is asking it to perform a computationally irreducible algorithm in zero substrate steps, which is impossible by definition. Asking it to emit the same algorithm step by step on a properly designed substrate works because each step is within the model's reliable per-token range.

Substrate-level Turing completeness, formalized Combining §2.3 (substrate properties) and §5.1–§5.2: the trace + slicer + model triple satisfies Turing-machine implementation requirements:

- Tape: trace + trim continuation (unbounded).
- Transition function: the model's forward pass (bounded local read, deterministic at temperature 0).
- Step semantics: token emission, byte-for-byte against the reference.
- Halt: `DONE` stop sequence.

The substrate is not the model alone; it is the model in combination with the tape design and the continuation primitive. Each substrate property is one piece of that implementation.

What's not in this paper

- We do not prove anything about model internals. Mechanistic interpretability of the relevant circuits is left to other work (Anthropic 2025 and follow-ups).
- We do not characterize per-class reliability ceilings quantitatively; the 12 failure modes are anecdotal, not systematically measured.
- We do not test beyond 128-digit operands. The substrate should scale; 256-digit is the obvious next step.
- We do not compare directly against fine-tuned approaches at matching scales; their published results stop earlier.

1.2.6 Related Work

Theoretical Turing completeness for transformers. Pérez et al. (2021), Li & Wang (2025), Merrill (2023). Position unchanged from v1.

Production-model arithmetic and the compositional-limit framing. Dziri et al. (2024), Ball et al. (2024), Stolfo et al. (2024). Position unchanged from v1.

Fine-tuned or specialized-architecture multiplication. - Wan et al. (2024): specialized tiny transformer, 99.9% on 5-digit multiplication. - Lee et al. (2024) "Teaching Arithmetic to Small Transformers" and follow-ups: 12–15 digit multiplication with reversed-digit order and custom embeddings; 100% on 100-digit addition with a single GPU-day of training. - McLeish et al. (2024) "Transformers Can Do Arithmetic with the Right Embeddings" (Abacus embeddings): addition main result, multiplication still 15 digits. - Deng et al. (2024) "From Explicit to Implicit CoT": GPT-2 Small with stepwise internalization reaches $\sim$ 100% on 9×9 multiplication. - Yu et al. (2025) "Why Can't Transformers Learn Multiplication?": reverse-engineering of ICoT models; identifies Fourier-basis representations of digits and Minkowski-sum pairwise-product caching; proposes an auxiliary running-sum probe loss. - "Chain of Thought in Order" (2025): rediscovers reverse-digit ordering via search over CoT token orders. - "How Numerical Precision Affects Arithmetical Reasoning" (2025): reverse output-digit order improves accuracy.

All of these operate via training or fine-tuning. None reach the operand scale demonstrated here on a generalist production model via in-context learning alone.

Mechanistic interpretability. Anthropic circuit tracing (2025), unchanged from v1.

Cellular automata and computational universality. Wolfram (2002), Cook (2004), unchanged from v1.

1.2.7 Conclusion

The v1 thesis stands: production transformers contain the computational machinery for universal computation, accessible via in-context learning. v2 extends the demonstration along three axes:

1. **Scale.** A long compositional algorithm executed end-to-end at 64 digits with $\sim$ 100% measured accuracy ($n = 25$ planned) and demonstrated at 128 digits ($\sim$ 4000 atomic steps) as an existence proof, on operand sizes substantially past any published in-context-learning result.
2. **Substrate primitive.** Trim continuation with landmark slicing removes the context-window bound, realizing the unbounded-tape primitive a Turing machine requires.
3. **Constructive evidence.** A 12-entry failure-mode cascade characterizes the per-token operations the model performs reliably, and the structural interventions that keep every per-token decision inside that reliable range.

The result reframes the published compositional limits on transformer arithmetic: they characterize a specific interface between task and attention, not a property of the architecture. Given an appropriate substrate, a production transformer trained without specialization can execute compositional arithmetic at scales far beyond what fine-tuning approaches have demonstrated, and at scales that make the universal-computation claim constructive rather than existential.

1.3 Tape snippet strategy

Same approach as v1: fenced code blocks in markdown (paperese 0.0.16+ auto-applies `scriptsize` to verbatim code blocks). For each snippet, include:

1. Short caption describing the step.
2. Snippet, 6–15 lines, truncated with **...** if longer.
3. Footnote / data-availability pointer to the full trace on GitHub Actions (specific run URL).

1.3.1 Specific snippets to include

Snippet A — Substrate primitives. RESUME, REFRESH, T block, OUT. From a small smoke test (8×8 or 12×12).

Snippet B — A single pair-step with T-table lookup. Show how leaves are anchored as `<digit>|<product>` and reference a T row.

Snippet C — Chained carry equation at row close. `row_sum + carry_in = total = carry_out*100 + cell`.

Snippet D — Final RETURN and DONE. Show wrapped RETURN tape + explicit halt marker.

Snippet E — Failure mode and its fix. Side-by-side: model's wrong emission vs the reference, with the substrate property that the fix introduced. Two clearest cases: - Carry slip (8818 → `c87`) before/after the chained equation. - Combine slip (`1|71 8|568 1278` → `1|71 8|568 781`) before/after the bracketed `P1*10+P2=prod` equation.

1.3.2 Linking out

Full traces in the GitHub Actions logs. Hard-link the run ID of the successful 128×128 in the data-availability appendix. For failure-mode illustrations, archive the relevant CI log excerpts to a `paper-artifacts/` directory in the repo before publication (we delete failed runs periodically).

1.4 Logistics

1. **Capture canonical CI artifacts before deletion.** For each failure-mode illustration, copy the relevant log excerpt to `paper-artifacts/`.
2. **Author list.** v1 was solo; v2 to be decided.
3. **Venue.** arXiv + Zenodo (DOI), as v1. Possibly a workshop submission (NeurIPS math-reasoning workshop, ICML).
4. **Length target.** v1 was ~7 pages; v2 likely ~12–15 with the failure-mode cascade in either main text or appendix.
5. **Reproducibility.** Repo URL + commit SHA of the headline run.

1.5 References (to cite or mention)

Full bibliography assembled across the v1 paper's reference list and the 2024–2025 literature surfaced for v2. Some entries are placeholders pending exact citation details at draft time.

1.5.1 Theoretical Turing completeness and complexity of transformers

- Pérez, Barceló, Marinkovic (2021). *Attention is Turing complete.* Journal of Machine Learning Research 22(75):1–35.
- Li, Wang (2025). *Constant bit-size transformers are Turing complete.* arXiv:2506.12027.
- Merrill (2023). *Saturated transformers are constant-depth threshold circuits.* TACL 11:1303–1317.
- Merrill, Sabharwal (2024). *The expressive power of transformers with chain of thought.* arXiv:2310.07923 (latest revision).

1.5.2 LLM arithmetic — compositional-limit framing

- Dziri, Lu, Sclar, Li, Jiang, Lin, Welleck, West, Bhagavatula, Le Bras, Hwang, Sanyal, Ren, Ettinger, Harchaoui, Choi (2024). *Faith and fate: Limits of transformers on compositionality.* NeurIPS 36.
- Ball, Chen, Herley (2024). *Can we count on LLMs? The fixed-effect fallacy and claims of GPT-4 capabilities.* arXiv:2409.07638.
- Stolfo, Belinkov, Sachan (2024). *Language models do hard arithmetic tasks easily and hardly do easy arithmetic tasks.* arXiv:2406.02356.
- Jain, Schwarzschild, Addepalli, Goldstein (2024). *IGC: Integrating a gated calculator into an LLM to solve arithmetic tasks reliably and efficiently.* arXiv:2501.00684.

1.5.3 LLM arithmetic — specialized / fine-tuned approaches

- Wan, Liu, He, Lin (2024). *Dissecting multiplication in transformers: Insights into LLMs.* AAAI 2024.
- Lee, Sreenivasan, Lee, Lee, Papailiopoulos (2024). *Teaching arithmetic to small transformers.* ICLR 2024.
- McLeish, Bansal, Stein, Jain, Kirchenbauer, Bartoldson, Kailkhura, Bhatele, Geiping, Schwarzschild, Goldstein (2024). *Transformers can do arithmetic with the right embeddings* (Abacus embeddings). arXiv:2405.17399.
- Deng, Choi, Shieber (2024). *From explicit CoT to implicit CoT: Learning to internalize CoT step by step.* arXiv:2405.14838.
- Yu et al. (2025). *Why can't transformers learn multiplication? Reverse-engineering reveals long-range dependency pitfalls.* arXiv:2510.00184.
- [Author?] (2025). *Chain of Thought in Order: Discovering learning-friendly orders for arithmetic.* ICML 2025; arXiv:2506.23875.
- [Author?] (2025). *How numerical precision affects arithmetical reasoning in LLMs.* Findings of ACL 2025.
- Jelassi, d'Ascoli, Domingo-Enrich, Wu, Li, Charton (2023). *Length generalization in arithmetic transformers.* arXiv:2306.15400.

1.5.4 Scratchpad / chain-of-thought as substrate

- Nye, Andreassen, Gur-Ari, Michalewski, Austin, Bieber, Dohan, Lewkowycz, Bosma, Luan, Sutton, Odena (2021). *Show your work: Scratchpads for intermediate computation with language models.* arXiv:2112.00114.
- Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le, Zhou (2022). *Chain-of-thought prompting elicits reasoning in large language models.* NeurIPS 35.

1.5.5 Mechanistic interpretability

- Anthropic (2025). *Circuit tracing: Revealing computational graphs in language models.* transformer-circuits.pub.

- Elhage, Hume, Olsson, Schiefer, Henighan, Kravec, Hatfield-Dodds, Lasenby, Drain, Chen, et al. (2022). *Toy models of superposition*. transformer-circuits.pub.
- Olsson, Elhage, Nanda, Joseph, DasSarma, Henighan, Mann, Askell, Bai, Chen, et al. (2022). *In-context learning and induction heads*. transformer-circuits.pub.

1.5.6 Cellular automata and computational universality

- Cook (2004). *Universality in elementary cellular automata*. Complex Systems 15(1):1–40.
- Wolfram (2002). *A New Kind of Science*. Wolfram Media. (cited for the 256-rule enumeration AND the computational-irreducibility framing in §5.4.)
- Berlekamp, Conway, Guy (1982). *Winning Ways for Your Mathematical Plays*, vol. 2. (background on universality in cellular automata generally — may or may not cite depending on framing.)

1.5.7 Predecessors of this work

- Lewis (2024). *Empirical Turing completeness in a production transformer via in-context learning*. Zenodo. DOI 10.5281/zenodo.10984995. (the v1 paper, this paper's predecessor.)

1.5.8 Possible additions, to revisit before submission

- Recent Anthropic / OpenAI model cards for the model versions used.
- Any post-2025 paper specifically on long-trace stability / context-window scaling in production models.
- Work on tape-format design for LLM computation generally (Schick et al. 2023 Toolformer is adjacent; possibly relevant).
- Cognitive-science framing of "reasoning as computation" — Wolfram's pieces, Dennett if going philosophical (probably not for this paper).

1.6 Build

`paperese` (globally installed) builds the markdown to PDF. Same flow as v1:

```
mkdir -p paper-build && cp /Users/lewis/.bun/install/global/node_modules/paperese/examples/arxiv-two-column/preprint.sty paper-build/
cd paper-build && paperese ../paper-v2-outline.md -o paper-v2-outline.pdf
```

Default format is PDF via `latexmk + xelatex`. Verbatim/code blocks get `scriptsize` applied automatically in paperese 0.0.16+. The YAML frontmatter from v1 (`paper-paperese.md`) carries over directly — copy it as the starting point and update `title`, `abstract`, `output`.